

Activity 01

Scenario: Corporate Employee Management System

A company needs a database to manage information about its employees and the departments they work in. Each employee is assigned to one department. The database must satisfy the following constraints when entering data.

Identified Tables & Constraints:

1. Department Table

- dept_id: Department ID (Must be the Primary Key)
- dept_name: Department Name (Cannot be empty / NOT NULL)

2. Employee Table

- emp_id: Employee ID (Must be the Primary Key)
- emp_name: Employee Name (Cannot be empty / NOT NULL)
- email: Email Address (Each employee must have a distinct email / UNIQUE)
- salary: Salary (Must be greater than Rs. 20,000 / CHECK)
- dept_id: Assigned Department (Must be a Foreign Key linking to the Department table)

Practice Questions

1. CREATE: Create a database named **CompanyDB**. Inside it, create the **Department** table with the given constraints (Primary Key and NOT NULL).
2. CREATE: Create the **Employee** table incorporating all the mentioned constraints (Primary Key, NOT NULL, UNIQUE, CHECK, and Foreign Key).

```
create database CompanyDB;

use CompanyDB;

create table Department (
    dept_id INT primary key,
    dept_name VARCHAR(50) not NULL
);

show tables;
```

```
create table Employee(  
    emp_id INT primary key,  
    emp_name VARCHAR(100) not null,  
    email VARCHAR(100) unique,  
    salary DECIMAL(10,2)CHECK(salary>20000),  
    dep_id INT,  
    foreign key(dep_id) references  
Department(dep_id)  
);  
  
desc Employee;
```

3. ALTER (ADD): Add a new column named **phone_number** with the data type VARCHAR(10) to the **Employee** table.

```
alter table Employee  
add phone_number VARCHAR(15);
```

4. ALTER (DROP): Remove the **phone_number** column that you added in the previous question from the **Employee** table.

```
ALTER TABLE Employee  
DROP COLUMN phone_number
```

5. INSERT: Insert two records into the **Department** table for the "IT" and "HR" departments.
6. INSERT: Insert records for three (3) employees into the **Employee** table. Ensure their salaries are strictly above 20,000 and their emails are unique.

7. **UPDATE:** Update the salary of the employee with **emp_id = 1** to 55,000.

```
UPDATE Employee
SET Salary = 55000
WHERE emp_id = 1;
```

8. **DELETE & SELECT:** An employee has left the company. Delete the record of the employee with **emp_id = 3**. After that, write a query to display all remaining employees in the **Employee** table.

```
DELETE FROM Employee
WHERE emp_id = 3;
```

Activity 02

Scenario: E-commerce Order Management System

An online retail store wants to track their registered customers and the orders they place. They need a database to store customer details and their corresponding order information. We will use MySQL Built-in functions to analyze this data.

Identified Tables & Constraints:

1. Customer Table

- **customer_id:** Unique identifier for the customer (Primary Key)
- **first_name:** First name of the customer (NOT NULL)
- **last_name:** Last name of the customer (NOT NULL)
- **reg_date:** The date the customer registered (Date format)

2. Orders Table

- **order_id:** Unique identifier for the order (Primary Key)
- **customer_id:** The customer who placed the order (Foreign Key linking to Customer)
- **order_date:** The date the order was placed (Date format)
- **total_amount:** The total bill amount (Decimal/Float)
-

Practice Questions

1. **CREATE:** Create a database named **RetailDB** and create the table with the given constraints.
2. **CREATE:** Create the **Orders** table with the mentioned constraints (including the Foreign Key).

3. **INSERT:** Insert at least 3 records into the **Customer** table and 3 records into the **Orders** table.

```
create database RetailDB;
use RetailDB ;
create table Customer(
    customer_id INT primary key,
    first_name VARCHAR(40) not null,
    last_name VARCHAR(40) not null,
    reg_date DATE
);
show tables;
create table Orders(
    order_id INT primary key,
    custoemr_id INT,
    order_date Date,
    total_amount DECIMAL(10,2),
    foreign key(custoemr_id) references
Customer(customer_id)
);
desc Orders;
INSERT INTO Customer (customer_id, first_name,
last_name, reg_date)
VALUES
(1, 'Nimal', 'Perera', '2023-01-15'),
(2, 'Kamal', 'Silva', '2023-05-22'),
(3, 'Saman', 'Kumara', '2023-11-08');
INSERT INTO Orders (order_id, custoemr_id,
order_date, total_amount)
VALUES
(101, 1, '2023-06-10', 15400.50),
(102, 2, '2023-07-25', 8250.75),
(103, 1, '2023-12-01', 24600.00);
```

4. Write a query to display the customer's first name and last name combined as a and make sure the full name is in UPPERCASE letters.

```
select UPPER(CONCAT(first_name, ' ', last_name))  
as Full_name  
from Customer;
```

5. Write a query to find all customers whose **first_name** has more than 4 characters.

```
select first_name  
from customer  
where LENGTH(first_name)>4;
```

6. Write a query to find the total revenue (sum of all **total_amount**) and the highest single order amount (maximum **total_amount**) from the **Orders** table.

```
select SUM(total_amount) as Total_Revenue,  
MAX(total_amount) as highest_Order  
from Orders;
```

7. Write a query to extract only the Year and the Month from the **order_date** for all orders. Display them as **Order_Year** and **Order_Month**.

```
select order_id, YEAR(order_date) as Order_Year,  
month(order_date) as order_Month  
from Orders;
```

8. The store is giving a 10% discount on all orders. Write a query to display the **order_id**, original **total_amount**, and the new discounted amount. Use the **ROUND()** function to round the discounted amount to 2 decimal places.

```
select order_id,  
       total_amount as Original_amount,  
       ROUND(total_amount*0.90) as Discount_amount  
from Orders;
```

Activity 03

Project Specification: TechAcademy Student Tracker

Client Brief: "TechAcademy," an IT training institute, needs a simple tracking table to manage the students enrolled in their coding bootcamps. You have been hired as a freelance Database Administrator to set up this system and extract specific reports.

Database Architecture: You need to create a single table named **Bootcamp_Students** inside a database named **TechAcademyDB**.

- **student_id**: Integer, Primary Key, and must automatically generate numbers (AUTO_INCREMENT).
- **first_name**: String (Max 50 chars), Cannot be null.
- **last_name**: String (Max 50 chars), Cannot be null.
- **course_name**: Must only accept three specific values: 'Python', 'Java', or 'Web Dev' (Use ENUM data type).

Course_name ENUM('python','Java','WebDev') NOT NULL

- **enrollment_date**: Date format.
- **score**: Integer. If no score is provided during data entry, it should automatically save as 0 (Use DEFAULT constraint).

Phase 1: Setup & Data Entry (DDL & DML)

1. **Database & Table Creation:** Create the **TechAcademyDB** database and the **Bootcamp_Students** table exactly as specified in the architecture above.
2. **Data Seeding:** Insert four (4) students into the table.

- *Note:* For at least one student, do not insert a value for the **score** column so you can test if the `DEFAULT` constraint works. (Since **student_id** is `AUTO_INCREMENT`, you don't need to manually insert it).

```
create database TechAcademyDB;
use TechAcademyDB;
create table Bootcamp_Students(
    student_id INT AUTO_INCREMENT primary key,
    first_name VARCHAR(50) not null,
    last_name VARCHAR(50) not null,
    course_name ENUM('python','java','Web Dev') not
null,
    enrollment_date DATE,
    score INT default 0
);
desc Bootcamp_Students;
INSERT INTO Bootcamp_Students (first_name,
last_name, course_name, enrollment_date, score)
VALUES
('Kasun', 'Perera', 'Python', '2023-08-01', 85),
('Ruwan', 'Silva', 'Web Dev', '2023-08-15', 78),
('Nadeesha', 'Kumari', 'Java', '2023-09-02', 92);
INSERT INTO Bootcamp_Students (first_name,
last_name, course_name, enrollment_date)
VALUES
('Ama1', 'Fernando', 'Python', '2023-09-10');
```

3. The management wants a list of all students enrolled in the 'Python' course. The results must be sorted by **enrollment_date** from the newest to the oldest (Descending).

```
select * from Bootcamp_Students
where course_name = 'Python'
order by enrollment_date desc;
```

4. Write a query to find out how many students are enrolled in *each* course. (The output should show the **course_name** and the **Student_Count**).

```
select course_name,
COUNT(student_id) as Student_count
from Bootcamp_Students
group by course_name;
```

5. Write a query to find the details of the student(s) who has the highest **score** in the entire academy. (Hint: Use a subquery in the *WHERE* clause, i.e., **WHERE score = (SELECT MAX(...))**).

```
select * from Bootcamp_Students
where score = (select MAX(score)
from Bootcamp_Students
);
```

6. The instructor decided to give 5 bonus marks to everyone. Write a query to add 5 to the current **score** of all students enrolled in the 'Web Dev' course.

```
update Bootcamp_Students
set score = score + 5
where course_name = 'Web Dev';
```

Activity 04

Scenario: CityBank Data Audit & Transactions

You are working as a Junior Data Analyst for "CityBank". The bank manager wants you to audit the customer accounts and perform a secure fund transfer between two accounts. You will use advanced filtering, conditional logic, and Transaction Control commands to complete this task.

Database Architecture: You need to create a single table named **Bank_Accounts** inside a database named **CityBankDB**.

- **acc_no**: Integer, Primary Key.
- **acc_name**: String (Max 100 chars), Cannot be null.
- **acc_type**: String (Max 20 chars) - e.g., 'Savings', 'Current'.
- **balance**: Decimal (10,2) - The amount of money in the account.

```
INSERT INTO Bank_Accounts (acc_no, acc_name,  
acc_type, balance)  
VALUES  
(101, 'Nimal Perera', 'Savings', 60000.00),  
(102, 'Saman Kumara', 'Current', 8000.00),  
(103, 'Sunil Shantha', 'Savings', 35000.00),  
(104, 'Kamal Silva', 'Savings', 120000.00),  
(105, 'Ruwanthi Dias', 'Current', 4500.00);
```

Setup & Data Entry

1. **CREATE & INSERT:** Create the **CityBankDB** database and the **Bank_Accounts** table. Then, insert five (5) dummy customer records with varying balances (e.g., 5000, 25000, 75000, etc.). Make sure at least two customers have names starting with the letter "S" (e.g., 'Saman', 'Sunil').
2. The manager wants to find all customers whose name starts with the letter "S". Write a query to fetch their details.

```
select * from bank_accounts
```

```
where acc_name like 'S%';
```

3. The bank wants to reward its wealthiest customers. Write a query to find the top 3 accounts with the highest balance.

```
select * from bank_accounts  
order by balance desc  
limit 3;
```

4. Generate a custom report showing **acc_name**, **balance**, and a new temporary column called **Account_Category**.
- If balance is greater than 50000, label it as '**Premium**'.
 - If balance is between 10000 and 50000, label it as '**Standard**'.
 - If balance is less than 10000, label it as '**Low Balance**'.

```
select  
    acc_name,  
    balance,  
    case  
        when balance > 50000 then 'Premium'  
        when balance between 10000 and 50000 then  
'Standard'  
        else 'Low Balance'  
    end  
as Account_category
```

```
from bank_accounts;
```

5. You need to transfer Rs. 5000 from Account Number 101 to Account Number 102. Since this is a sensitive operation, write the SQL code using **START TRANSACTION**, perform the two necessary **UPDATE** statements (deducting from 101 and adding to 102), and finally **COMMIT** the changes to save them permanently.

```
start TRANSACTION;  
update bank_accounts  
set balance = balance - 5000  
where acc_no = 101;  
update bank_accounts  
set balance = balance + 5000  
where acc_no = 102;  
commit;  
select * from bank_accounts;
```

Activity 05

A software company needs a database to manage employees, projects, and their assignments. You are required to design and manipulate the database structure using SQL Data Definition Language (DDL) commands

Table Creation and Constraints:

1. **Create an Employee table** with the following columns and constraints:
 - emp_id: Primary Key
 - emp_name: NOT NULL
 - email: UNIQUE
 - phone
 - salary

- gender
- department
- job_title
- hire_date
- date_of_birth
- join_date
- address
- status: (Must store either 'active' or 'inactive')

2. **Create a Project table** with the following columns and constraints:

- project_id: Primary Key
- project_name: NOT NULL
- description
- start_date
- end_date
- budget
- status
- client_name
- project_manager
- created_at

3. **Create an Assignment table** with the following columns and constraints:

- Assignment_id: Primary Key
- emp_id: Foreign Key referencing the Employee table
- project_id: Foreign Key referencing the Project table
- assigned_date

Table Alterations and Management:

1. **Add a new column** named phone to the Employee table, ensuring it has a UNIQUE constraint. (Write the SQL statement)
2. **Modify the Project table** to add a new column named start_date which must be defined as NOT NULL. (Write the SQL statement)
3. **Rename the Assignment table** to Work_Assignment. (Write the SQL statement)
4. **Remove the column** phone from the Employee table. (Write the SQL statement)
5. **Delete the Assignment table** (which is now named Work_Assignment). (Write the SQL statement)

